SpIDER: Spatially Informed Dense Embedding Retrieval for Software Issue Localization

Anonymous Authors[1]

Abstract

Retrieving code functions, classes or files that are relevant in order to solve a given user query, bug report or feature request from large codebases is a fundamental challenge for Large Language Model (LLM)-based coding agents. Agentic approaches typically employ sparse retrieval methods like BM25 or dense embedding strategies to identify semantically relevant units. While embedding-based approaches can outperform BM25 by large margins, they often don't take into consideration the underlying graph-structured characteristics of the codebase. To address this, we propose SpIDER (Spatially Informed Dense Embedding Retrieval), an enhanced dense retrieval approach that integrates LLM-based reasoning along with auxiliary information obtained from graph-based exploration of the codebase. We further introduce SpIDER-Bench, a graph-structured evaluation benchmark curated from SWE-PolyBench, SWEBench-Verified and Multi-SWEBench, spanning codebases from Python, Java, JavaScript and TypeScript programming languages. SpIDER's graph-based candidate expansion gives each surfaced function a structural reason for inclusion (the seed it neighbors and the edge type linking them), making the candidate set auditable by a developer reviewing the agent's actions while keeping the retrieval budget fixed. The code graph is built from per-repository syntax trees, so it can be constructed on-demand at the start of an interactive developer session rather than precomputed offline across an entire codebase corpus. Empirical results show that SpIDER consistently improves dense retrieval performance by at least 13% across programming languages and benchmarks in SpIDER-Bench [1].

1. Introduction

Recent work has demonstrated the potential of agents powered by Large Language Models (LLMs) to perform automated bug repair and implement basic features within large-scale software repositories (Yang et al., 2024; Gauthier, 2024; Xia et al., 2024). A critical prerequisite for effective code generation is the precise identification of relevant contextual information, specifically code units such as functions, classes, or files. This code localization task is a foundational yet notably difficult challenge, particularly at the fine-grained level of function retrieval (Jimenez et al., 2024; Rashid et al., 2025; Zan et al., 2025; Chen et al., 2025). Poor code retrieval directly compromises generated code, leading to incorrect or incomplete fixes, new bugs, and increased computational cost and repair duration. Effective code localization is therefore essential not only for autonomous patch generation, but also for *developer oversight*: when retrieval is imprecise or its choices are unexplained, reviewers cannot verify what the agent considered, eroding developer trust.

Current approaches to code localization fall into two primary categories. Some methods exploit graph representations of code repositories alongside the reasoning capabilities of LLMs (Ouyang et al., 2025; Chen et al., 2025; Jiang et al., 2025). These approaches mainly employ sparse retrieval strategies such as BM25 (Robertson et al., 1994) to identify relevant graph nodes or subgraphs for agent exploration. Other methods train bimodal encoders using contrastive objective to semantically align dense embeddings of code units with issue descriptions (Fehr et al., 2025; Reddy et al., 2025). These dense retrieval methods rank code units based on their embedding similarity to the issue description.

These existing approaches face three critical limitations. First, conventional contrastive learning-based retrieval methods overlook the structural context of code modules within a repository when ranking them (Reddy et al., 2025; Zhang et al., 2024; Suresh et al., 2025). We observe that in software issue localization, buggy code modules are often spatially proximate within the codebase (e.g., within the same file

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

[1]Due to the substantial size of the SpIDER-Bench graphs, we provide two samples in the supplementary materials for reviewers to examine. The complete dataset will be released upon acceptance.

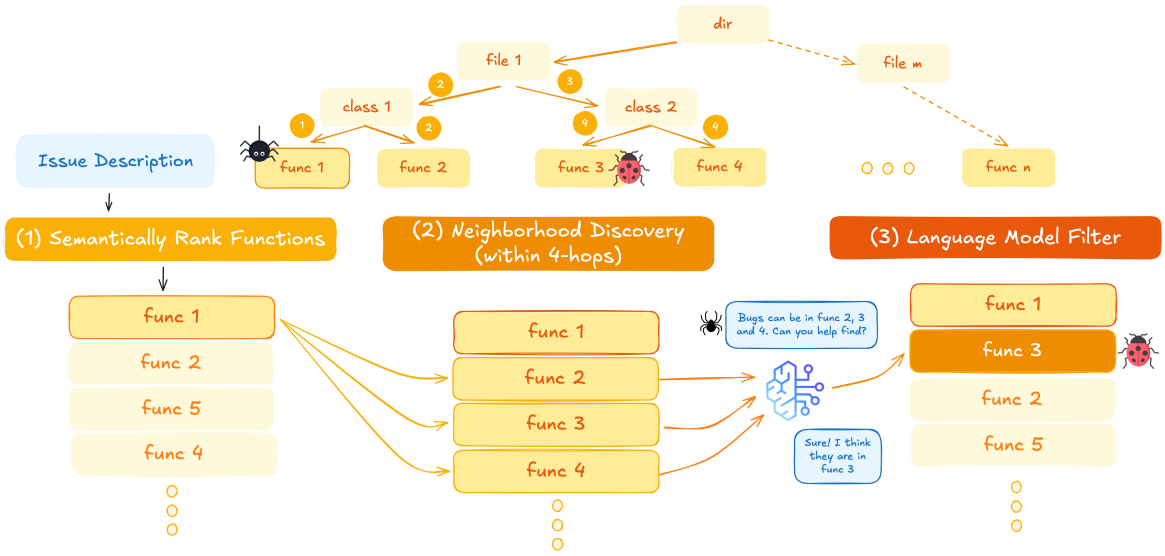

Figure 1. **SpIDER workflow.** SpIDER augments standard dense embedding retrieval with graph-aware spatial exploration and LLM-based reasoning to improve function-level code localization under a fixed retrieval (K). Given an issue description, all functions are first ranked by semantic similarity, and the top-K candidates are retrieved. The top-C (C<K) functions serve as centers (func 1) for structured neighborhood exploration along contains edges in the code graph, where neighboring functions within d hops are considered if they also rank within the top-N semantically (funcs 2,3,4). An LLM then filters these candidates to identify functions that are likely relevant but under-ranked by semantic similarity alone (func3). The selected neighbors are inserted immediately below their corresponding centers while discarding an equal number of bottom ranked functions in the initial top-K list, thereby increasing coverage of structurally related buggy functions without increasing K. Further algorithmic details are provided in Section A.8 (Algorithm 1).

or in neighboring functions), and top-ranked entities retrieved via dense semantic similarity frequently reside *near* the actual buggy modules. Consequently, relying exclusively on semantic similarity to select the top-K functions may overlook relevant candidates that are spatially close to highly-ranked modules but exhibit *marginally* lower embedding similarity scores. Second, existing methods have been primarily evaluated on Python (Chen et al., 2025; Fehr et al., 2025), particularly `SWEBench-Lite`, `SWEBench-Verified`, and `LocBench` (Jimenez et al., 2024; Chowdhury et al., 2024b; Chen et al., 2025) – leaving their performance on other programming languages largely underexplored. Third, function-level retrieval, which represents the most challenging granularity compared to class-level and file-level retrieval(Rashid et al., 2025; Reddy et al., 2025; Chen et al., 2025), remains largely understudied.

These limitations matter more in interactive, developer-in-the-loop settings, where the retrieval step must run cheaply on a developer's active repository at session start and expose its choices to the reviewer. Function-level granularity matches how developers reason about edits. The graph-based candidate expansion in `SpIDER` also attaches a structural reason for inclusion to each surfaced function, namely its position relative to a top-ranked seed, rather than only an embedding score. The code graph is built on-demand from per-repository syntax trees, avoiding the offline corpus-wide preprocessing of many graph-based pipelines.

To address these limitations, we introduce `SpIDER`, a simple graph-aware dense retrieval strategy for *function-level* code localization. `SpIDER` incorporates spatial locality as a complementary signal to semantic similarity, enabling the joint exploitation of both code content and repository structure within a fixed retrieval budget. Our primary focus is on retrieval methods that can augment downstream tasks, including ranking code modules, precise localization, and automated patch generation to resolve GitHub-style issues. To facilitate comprehensive evaluation across multiple programming languages, we also introduce `SpIDER-Bench` for code localization, encompassing Python, Java, JavaScript, and TypeScript repositories from `SWEBench-Verified`, `SWE-PolyBench`, and `Multi-SWEBench`.

Our primary contributions are as follows: (1) We propose `SpIDER`, a novel and simple graph-aware dense retrieval strategy combining semantic content and code structure to identify the most relevant functions for a given issue under a fixed retrieval budget. (2) We introduce `SpIDER-Bench`, a heterogeneous graph-structured benchmark for multi-language code localization, built from GitHub repositories in `SWEBench-Verified`, `SWE-PolyBench`, and `Multi-SWEBench` covering Python, Java, JavaScript, and TypeScript, with source code as node-level features. We comprehensively validate `SpIDER` against existing dense retrievers and the popular sparse method `BM25` across all four languages in `SpIDER-Bench`.

2. Related Work

Code Retrieval and Software Issue Localization Code retrieval refers to identifying code locations responsible for a software issue. Classical approaches are rooted in information retrieval, leveraging lexical or semantic similarity to rank candidate code snippets. Consequently, many existing systems rely on sparse retrievers such as BM25 (Robertson et al., 1994). While BM25 indexing is cheaper than dense embedding generation and vector similarity search, it typically underperforms when rich semantic representations are available (Reddy et al., 2025; Fehr et al., 2025).

Recent dense retrieval approaches propose bimodal encoder models to encode both code chunks as well as issue embeddings in an aligned latent space (Zhang et al., 2024; Suresh et al., 2025; Feng et al., 2020; Guo et al., 2021). However, these models are typically pretrained on generic natural language–to–code objectives, which leads to suboptimal performance for issue-driven code retrieval. To address this gap, Fehr et al. (2025) and Reddy et al. (2025) introduce bimodal encoders fine-tuned on GitHub-style issue resolution datasets such as `SWEBench` (Jimenez et al., 2024), resulting in improved alignment between issue and code representations. Despite these gains, `SWEBench` is limited to Python repositories, and no comparably large multilingual dataset currently exists to support training such encoders across diverse programming languages. Consequently, as real-world codebases span multiple languages, these fine-tuned dense retrievers exhibit degraded performance under domain shifts, reducing their robustness and reliability. *We argue that incorporating the commonsense reasoning capabilities of large language models (LLMs), together with structural signals derived from code graphs, introduces more invariant representations that are less sensitive to surface-level language artifacts*. Augmenting dense retrievers with these components helps mitigate spurious correlations and improves generalization. We empirically validate this hypothesis on multilingual benchmarks, including `SWE-PolyBench` (Rashid et al., 2025), `Multi-SWEBench` (Zan et al., 2025), and `SWEBench-Verified` (Chowdhury et al., 2024a).

LLM-based retrieval methods Recent work has shown that large language models (LLMs) are highly effective at localizing and resolving GitHub-style software issues, largely due to their strong reasoning capabilities (Kang et al., 2024; Xia et al., 2024; Luo et al., 2024; Örwall, 2024). In the context of code retrieval, Xia et al. (2024) introduce a simple hierarchical localization strategy driven directly by an LLM, whereas Ouyang et al. (2025) employ an intermediate sparse BM25 retrieval stage to identify relevant files prior to code generation. Similarly, Chen et al. (2025) leverage inverted BM25 indexing to match issue-description keywords with node identifiers or code chunks. Both Ouyang et al. (2025) and Chen et al. (2025) further enable LLM-guided traversal of local subgraphs around the retrieved files or nodes, using edges defined by their respective code graph representations. In contrast, Reddy et al. (2025) focus on re-ranking densely retrieved code chunks using LLMs, with the goal of enhancing semantic retrieval through reasoning. Although this approach improves retrieval performance, it does not exploit the structural signals encoded in code graphs. *We aim to bridge this gap by jointly leveraging LLM reasoning, semantic embeddings, and the additional contextual information captured by code graphs, thereby improving overall retrieval effectiveness.*

Code Graph Construction Several recent approaches leverage explicit code graph representations to expose structural information that is otherwise difficult to capture with purely textual or embedding-based methods. The specific utility of such graphs depends critically on how nodes and relations are defined. For example, Ouyang et al. (2025) construct graphs at the line-of-code level, where nodes correspond to variable or module definitions and references, and edges encode *invokes* and *contains* relationships. While this design enables fine-grained structural reasoning, it scales poorly with repository size, leading to dense and complex graphs that are challenging for LLM-based agents to traverse at inference time.

To improve scalability, other works adopt coarser graph abstractions. Jiang et al. (2025) propose two complementary graph types: a module call graph, connecting files via *imports*, and a function call graph, linking functions and classes through *invokes* and *inherits* relations. More recently, Chen et al. (2025) introduce a unified graph formulation in which nodes represent functions, classes, files, or directories, and edges capture *contains*', *invokes*', *imports*', and '*inherits*' relationships. This design strikes a balance between expressivity and tractability, enabling structural reasoning at multiple hierarchical levels while remaining amenable to LLM-guided traversal. *Following this line of work, we adopt their graph construction, as it naturally supports retrieval at the file, class, and function levels—granularities that are common across programming languages and well aligned with multilingual code retrieval settings.*

3. Problem Definition and Notations

We formalize the function-level code retrieval task and its evaluation, applicable to any retrieval method operating over structured code representations.

Given a codebase and an issue description Q, the goal is to retrieve the top-K functions that are most likely to require edits to resolve the issue. We represent the codebase as a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ with n nodes, where $\mathcal{V} = v_{i}{}_{i=1}^{n}$ denotes the set of code entities and $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ encodes structural

relations. Each node $v \in \mathcal{V}$ corresponds to a code entity, such as a function, class, file, or directory; and edges represent relationships of types *contains*, *invokes*, *imports*, and *inherits*. Let $\mathcal{V}^* = v_{i\,i=1}^{*m} \subseteq \mathcal{V}$ denote the ground-truth set of m relevant function nodes for issue Q.

Scoring Function A bi-modal encoder $\mathcal{F}(\cdot)$ maps both code units and issue descriptions to a shared embedding space. The relevance score of node v for issue Q is:

$$s_Q(v) = \cos\big(\mathcal{F}(v), \mathcal{F}(Q)\big)$$

Baseline Dense Retrieval Standard dense embedding retrieval (DER) ranks all nodes by their relevance scores and returns the top-K candidates:

$$\mathcal{S}_K(Q) := \arg\operatorname{top}_{K\ v\in\mathcal{V}}\ s_Q(v)$$

where $\arg\operatorname{top}_K$ returns K nodes with top-K scores $s_Q(\cdot)$. We use notation $\mathcal{S}_K$ and $\mathcal{S}_K(Q)$ interchangeably.

4. Methodology

4.1. Building Code Graphs

For a given codebase, we construct a code graph using only files written in the primary programming language (the language comprising the majority of the repository). Files in other languages are ignored to simplify construction; e.g., in a Python repository, we consider only Python source files.

We follow the graph schema proposed by Chen et al. (2025), where each node represents a code entity—function, class, file, or directory—and edges encode structural relations of types *imports*, *invokes*, *contains*, and *inherits*. For Python, we use the built-in `ast` module[2] to parse source files and extract syntax trees. For Java, JavaScript, and TypeScript, we rely on Tree-sitter[3] for syntax parsing.

Graph construction proceeds by traversing the repository hierarchy. Directory nodes are added to enable navigation across the project structure. File nodes are included only if they contain at least one function or class definition; files containing solely metadata or configuration are ignored. Function and class nodes, along with the edges connecting them, are extracted from the corresponding syntax trees. While directory nodes do not store code content, all other node types include their associated source code.

Constructing accurate graphs for Java, JavaScript and TypeScript presents additional challenges due to the flexible ways in which functions and classes can be declared/implemented. For example, class definitions and method implementations may be split across multiple locations or files. We explicitly handle such language-specific cases using Tree-sitter to ensure faithful graph construction. Refer App. A.9.

Although the resulting graph contains multiple node types, we focus exclusively on retrieving *functions*, which constitute the most granular, and consequently the most challenging retrieval unit. This choice is motivated by statistics from the `SWE-PolyBench` dataset, where 67% of instances involve function-level edits, with an average of 2.78 edits per instance (Rashid et al., 2025). In contrast, only 1.42% of instances involve class-only edits (excluding method changes), with an average of 0.49 edits per instance. Since files are composed of functions and classes, accurate retrieval at finer granularities naturally helps file-level localization as well.

Table 1 reports dataset statistics, including the proportion of retained instances per language. For function-level retrieval, we exclude instances involving only file- or class-level edits. Similarly, for class- and file-level retrieval, we ignore instances with edits exclusively at other granularities.

4.2. Leveraging Code Graph via SpIDER

Figure 1 provides a high-level overview of SpIDER, with algorithmic details presented in Algorithm 1 (Section A.8). Given an issue description Q and a codebase represented as graph $\mathcal{G}$, SpIDER retrieves the top-K functions most likely to require modification. While we focus on function-level retrieval, the framework naturally extends to class- and file-level retrieval by aggregating subsets of code chunks when full contexts exceed the LLM budget (see Section A.10).

SpIDER's design is motivated by the observation that, in instances requiring multiple edits, relevant functions tend to be structurally proximate in the graph due to call dependencies and containment relationships. While dense semantic retrieval provides broad coverage, it often fails to capture "near-miss" cases, where the correct function lies close *structurally* but not *semantically* to top-ranked candidates.

Semantic Retrieval We first compute the baseline dense retrieval $\mathcal{S}_K(Q)$ as defined in Section 3. Additionally, we compute $\mathcal{S}_N(Q)$ containing the top-N nodes ($N > K$) to constrain neighborhood exploration.

Seed Selection From $\mathcal{S}_K(Q)$, we select the top-C ranked nodes as seed centers for graph exploration:

$$\mathcal{C}_Q := v \in \mathcal{S}_C(Q)$$

where $C \leq K$. A seed anchors a local neighborhood search.

Neighborhood Exploration For each seed, we perform breadth-first search up to depth d along 'contains' edges, collecting structurally proximate function nodes. Formally, the d-hop neighborhood is:

$$\Gamma_d(\mathcal{C}_Q) := \{u \in \mathcal{V} \mid \operatorname{dist}_{\mathcal{G}}(u, \mathcal{C}_Q) \leq d\}$$

[2] https://docs.python.org/3/library/ast.html
[3] https://github.com/tree-sitter

Table 1. **SpIDER-Bench data statistics after graph construction (GT = ground truth).** It summarizes the distribution and granularity of ground-truth edits across languages and datasets, showing that function-level edits dominate across benchmarks while exhibiting substantial variation in edit frequency and localization difficulty across programming languages.

GT Type	Original Dataset	Python		Java		JavaScript		TyeScript	
		% Insts. with GT edits	Avg. GT edits per inst.	% Insts. with GT edits	Avg. GT edits per inst.	% Insts. with GT edits	Avg. GT edits per inst.	% Insts. with GT edits	Avg. GT edits per inst.
Function	SWE-PolyBench	85.93	3.73	70.30	5.52	53.98	1.98	29.63	2.68
	Multi-SWEBench	N/A	N/A	59.54	2.46	81.74	3.88	27.68	2.29
	SWEBench-Verified	89.00	1.82	N/A	N/A	N/A	N/A	N/A	N/A
Class	SWE-PolyBench	75.38	2.9	87.27	3.07	20.45	1.37	19.07	1.91
	Multi-SWEBench	N/A	N/A	81.68	1.58	9.55	1.06	3.13	1.57
	SWEBench-Verified	74.00	1.40	N/A	N/A	N/A	N/A	N/A	N/A
File	SWE-PolyBench	99.50	1.91	100.00	3.2	58.31	1.68	55.97	2.00
	Multi-SWEBench	N/A	N/A	97.71	1.56	98.60	2.37	70.54	5.08
	SWEBench-Verified	100.00	1.24	N/A	N/A	N/A	N/A	N/A	N/A

where $\text{dist}_{\mathcal{G}}(u, \mathcal{C}_Q)$ denotes the shortest-path distance from u to any node in $\mathcal{C}_Q$. Two functions within the same class are 2 hops apart; functions in different classes across separate files are 4 hops apart. While we use 'contains' edges to capture hierarchical structure, the framework can incorporate other edge types like 'invokes', 'imports', and 'inherits'.

Two-Stage Neighbor Filtering

To control computational cost while maintaining quality, candidates are filtered in two stages. *Stage 1 (Semantic Filtering):* We restrict neighbors to those also appearing in the top-N candidate pool:

$$\hat{\mathcal{C}}_Q := \Gamma_d(\mathcal{C}_Q) \cap \mathcal{S}_N(Q)$$

This ensures exploration remains within semantically relevant regions of the graph. *Stage 2 (LLM-based Selection):* An LLM selector $\mathcal{L}$ evaluates each candidate $u \in \hat{\mathcal{C}}_Q$, receiving the source code content and issue description as context. The LLM returns a binary relevance decision $\mathcal{L}(u) \in \{0, 1\}$. Note that seed centers $\mathcal{C}_Q$ are always retained in $\mathcal{U}_Q$, where:

$$\mathcal{U}_Q := u \in \hat{\mathcal{C}}_Q \mid \mathcal{L}(u) = 1 \cup \mathcal{C}_Q$$

Output Construction To maintain a fixed retrieval budget of K, newly selected neighbors replace the lowest-ranked non-center nodes. The final retrieval set is:

$$\mathcal{S}_K^{\text{LLM}}(Q) := \mathcal{U}_Q \cup \underset{v \in \mathcal{S}_K(\mathcal{Q})}{\arg\text{top}_{K-|\mathcal{U}_Q|}} \; s_Q(v) \qquad (1)$$

Hyperparameters SpIDER introduces four hyperparameters: retrieval budget K, number of seed centers C, exploration depth d, and semantic filtering threshold N. We analyze their effects in Section 5.3.

Definition 4.1 (Expected Recall Change)**.**

$$\pi_B := \Pr(u \in \mathcal{V}^* \mid u \in \mathcal{S}_K \setminus \mathcal{S}_{K-|\mathcal{U}_Q|}),$$
$$\pi_\Gamma := \Pr(u \in \mathcal{V}^* \mid u \in \hat{\mathcal{C}}_Q \setminus \mathcal{S}_K).$$

Proposition 4.2 (Sufficient Condition for Recall Improvement)**.** *Following the definitions from sections 3 and 4.2 and under the assumptions 1, 2 and 3 listed in Section A.2, if it holds that* $\alpha\pi_\Gamma > \pi_B + \beta(1 - \pi_\Gamma)$*, then we can say,*

$$\mathbb{E}[\text{Rec@}K(\mathcal{S}_K^{\text{LLM}})] > \mathbb{E}[\text{Rec@}K(\mathcal{S}_K)].$$

Proposition 4.2 shows that when (i) relevant functions form a localized region in the code graph, (ii) dense retrieval places at least one seed in that region, and (iii) the LLM selector has higher true-positive than false-positive rate, then graph-aware retrieval strictly improves expected recall at fixed budget K. Proof can be found in Section A.1, and Section A.7 provides an empirical analysis on SWEBench-Verified confirming that the graph-locality precondition (Assumption 2) holds in practice: multi-edit instances cluster within a few hops, and most ground-truth pairs lie at distance ≤ 4.

5. Experiments and Results

We design our experiments to investigate whether leveraging structural relationships within code improves retrieval performance, particularly in challenging function-level settings. We focus on three high-level research questions:

RQ1 (Exploiting Graph Structure) Can incorporating the structural relationships between code modules improve retrieval of relevant functions beyond what semantic similarity alone achieves?

RQ2 (Cross-Language Robustness) Do retrieval strategies that leverage code structure generalize across programming languages and codebases of varying complexity?

RQ3 (Efficiency and Practicality) Can graph-aware retrieval methods achieve improved coverage without incurring excessive computational cost or LLM invocations?

5.1. Experimental Setup

Datasets We focus on challenging multi-function edits and evaluate retrieval across multiple languages. We use SWEBench-Verified, SWE-PolyBench, and Multi-SWEBench, with patched functions as ground truth,

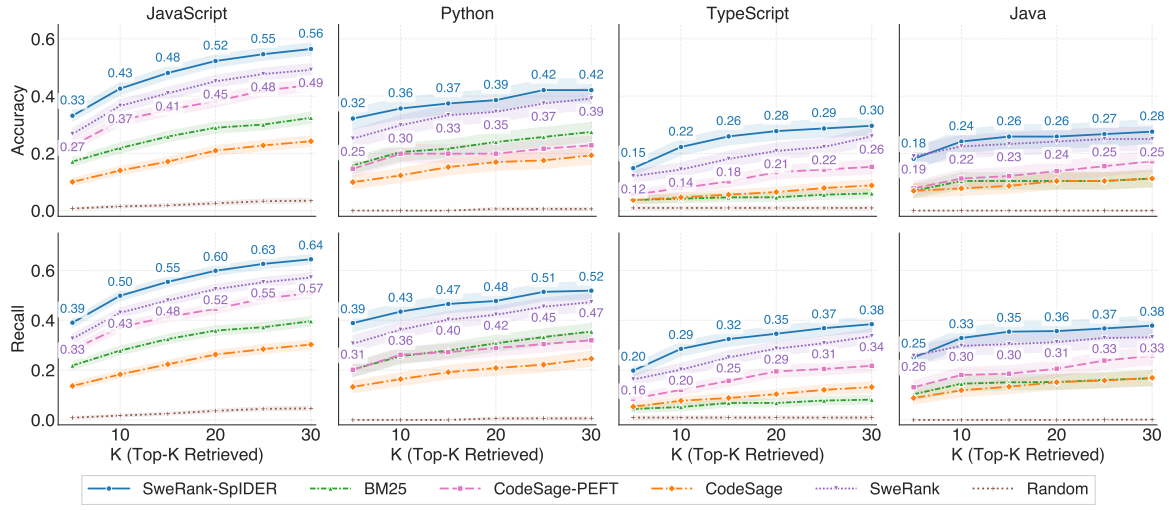

Figure 2. **Function-level retrieval performance on `SWE-PolyBench`.** Across all programming languages and retrieval budgets K, SWERankEmbed-Small+`SpIDER` consistently outperforms dense and sparse baselines in both Accuracy and Recall, highlighting robust gains from incorporating graph-aware neighborhood expansion under a fixed retrieval budget. Shaded envelopes indicate standard deviations estimated via bootstrapping with 1000 draws.

and following Suresh et al. (2025) exclude instances without function-level modifications. The evaluation spans Python, Java, TypeScript, and JavaScript repositories, assessing both the robustness of our approach and the generalization of baselines across languages and codebase complexities.

Evaluation Metrics Given a query Q and a retrieved set $\mathcal{S}(Q)$ of size K, we evaluate retrieval quality using Recall@K and Acc@K, defined as

$$\text{Recall@}K = \frac{|\mathcal{V}^* \cap \mathcal{S}(Q)|}{|\mathcal{V}^*|}, \qquad \text{Acc@}K = \mathbb{I}[\mathcal{V}^* \subseteq \mathcal{S}(Q)]$$

where $\mathcal{V}^*$ denotes the set of ground-truth patched functions.

Recall@K measures the fraction of ground-truth functions recovered within the top-K retrieved results, while Acc@K is a stricter metric that equals 1 only when *all* ground-truth functions are included in the top-K set. These metrics are particularly well-suited to the multi-function setting, as they directly capture coverage under a fixed retrieval budget. For completeness, we additionally report Mean Reciprocal Rank (MRR) in App. Section A.11, defined as the reciprocal of the rank at which any ground-truth function first appears in the retrieved list.

Hyperparameters We analyze hyperparameters C, d, N, and K in Section 5.3. Following prior work (Reddy et al., 2025; Fehr et al., 2025), we fix $K = 20$ across datasets to ensure fair comparison with baselines sensitive to this choice (e.g., embedding-based retrieval with LLM reranking). The remaining `SpIDER` hyperparameters are selected via grid search on the Python split of `SWE-PolyBench`, yielding $C = 5$, $d = 4$, $N = 500$; we hold these constant across all experiments to avoid overfitting. Our ablations further examine how these parameters affect performance and practical considerations such as LLM context length and invocation budget. We use `Claude Sonnet 4` with temperature 0.1 for consistent and reproducible evaluation. Qwen Coder 30B Instruct results are also reported in Table 16.

Baselines We compare `SpIDER` against representative state-of-the-art retrieval approaches spanning both dense and sparse methods. For dense retrieval, we consider `SweRankEmbed-Small ZS` and `CodeSAGE-Small ZS`, which are widely used embedding-based models for software issue localization and code retrieval (Reddy et al., 2025; Zhang et al., 2024). As a sparse retrieval baseline, we include BM25 (Robertson et al., 1994).

Since `CodeSAGE-Small ZS` is not trained for GitHub-style problem-solving tasks, we additionally evaluate a parameter-efficiently fine-tuned variant using LoRA (Hu et al., 2022) on 2000 instances from three repositories in the `SWEBench` training set, denoted as `CodeSAGE-Small-PEFT`.

Some recent methods could not be fully evaluated. Fehr et al. (2025) did not release their embedding model or code, preventing direct comparison. `LocAgent` (Chen et al., 2025) is based on a time consuming and iterative LLM-driven exploration and is therefore evaluated only on `SWEBench-Verified` and the Python repositories of `SWE-PolyBench` in tables 8 and 9.

5.2. Results

Leveraging code graphs improves retrieval Table 2 demonstrates the benefit of incorporating code graph

Table 2. **Main result.** Across all embedding models and programming languages, SpIDER consistently improves Recall and Accuracy over dense retrieval alone, with gains preserved and often amplified after LLM reranking. Here, $K = 20$ with $N = 500$, $C = 5$, and $d = 4$; best results per embedding model are highlighted in light blue. SpISR denotes the spatially informed sparse retriever analogous to SpIDER. KDE plots are provided in Figs. 4 and 5 while confidence intervals are provided in Table 10.

Language	Model	Retrieval	SWE-PolyBench				SWEBench-Verified	
			Retrieval Only		Retrieval + LLM reranker		Retrieval Only	
			Recall@20	Accuracy@20	Recall@3	Accuracy@3	Recall@20	Accuracy@20
Python	SweRankEmbed-Small ZS	SpIDER	0.49	0.40	0.44	0.36	0.61	0.56
		DER	0.42	0.35	0.38	0.31	0.54	0.49
	CodeSAGE-Small ZS	SpIDER	0.27	0.21	0.23	0.18	0.30	0.27
		DER	0.21	0.17	0.19	0.16	0.21	0.19
	CodeSAGE-Small-PEFT	SpIDER	0.37	0.27	0.32	0.25	0.55	0.49
		DER	0.29	0.20	0.26	0.20	0.43	0.38
	BM25	SpISR	0.33	0.26	0.30	0.23	0.45	0.41
		SR	0.31	0.24	0.27	0.22	0.38	0.34
							Multi-SWEBench	
Java	SweRankEmbed-Small ZS	SpIDER	0.36	0.27	0.25	0.17	0.37	0.28
		DER	0.31	0.24	0.24	0.16	0.32	0.24
	CodeSAGE-Small ZS	SpIDER	0.23	0.17	0.16	0.09	0.21	0.19
		DER	0.15	0.10	0.11	0.07	0.15	0.13
	CodeSAGE-Small-PEFT	SpIDER	0.28	0.18	0.20	0.14	0.31	0.25
		DER	0.21	0.14	0.17	0.11	0.23	0.17
	BM25	SpISR	0.16	0.10	0.12	0.08	0.23	0.17
		SR	0.15	0.10	0.12	0.09	0.22	0.17
JavaScript	SweRankEmbed-Small ZS	SpIDER	0.60	0.52	0.43	0.37	0.39	0.30
		DER	0.52	0.45	0.41	0.34	0.31	0.23
	CodeSAGE-Small ZS	SpIDER	0.38	0.32	0.30	0.25	0.17	0.12
		DER	0.26	0.21	0.22	0.17	0.09	0.06
	CodeSAGE-Small-PEFT	SpIDER	0.55	0.48	0.41	0.35	0.32	0.24
		DER	0.45	0.38	0.36	0.30	0.25	0.18
	BM25	SpISR	0.46	0.39	0.36	0.30	0.21	0.14
		SR	0.36	0.29	0.31	0.25	0.14	0.10
TypeScript	SweRankEmbed-Small ZS	SpIDER	0.35	0.28	0.27	0.21	0.52	0.42
		DER	0.29	0.21	0.25	0.19	0.40	0.32
	CodeSAGE-Small ZS	SpIDER	0.16	0.10	0.14	0.09	0.33	0.26
		DER	0.10	0.06	0.09	0.06	0.26	0.21
	CodeSAGE-Small-PEFT	SpIDER	0.25	0.18	0.22	0.16	0.30	0.26
		DER	0.19	0.13	0.18	0.12	0.19	0.15
	BM25	SpISR	0.11	0.09	0.10	0.07	0.32	0.27
		SR	0.07	0.05	0.07	0.05	0.24	0.21

Table 3. **Ablation on number of seed centers** C using SweRankEmbed-Small+SpIDER. Increasing C improves Recall@20 and Accuracy@20 across languages up to saturation.

C	Python		JavaScript		Java		TypeScript	
	Recall@20	Acc@20	Recall@20	Acc@20	Recall@20	Acc@20	Recall@20	Acc@20
1	0.47	0.38	0.56	0.48	0.32	0.24	0.32	0.24
3	0.49	0.39	0.59	0.51	0.35	0.26	0.31	0.24
5	0.49	0.40	0.60	0.52	0.36	0.27	0.35	0.28
7	0.49	0.39	0.60	0.52	0.36	0.27	0.37	0.30
9	0.48	0.39	0.60	0.53	0.37	0.27	0.39	0.31

structure into retrieval. Comparing standard dense (DER) and sparse (SR) retrieval with our graph-aware variant (SpIDER) across Python, TypeScript, JavaScript, and Java repositories from SWE-PolyBench, Multi-SWEBench, and SWEBench-Verified at $K = 20$, we observe consistent improvements across all datasets and languages.

Fig. 2 further illustrates this trend on SWE-PolyBench as the retrieval budget K varies. SweRankEmbed-Small + SpIDER consistently outperforms all baselines across languages and K values, highlighting the effectiveness of combining semantic similarity with explicit modeling of spatial relationships and LLM-based neighborhood filtering. Notably, the zero-shot SweRankEmbed-Small model—trained exclusively on roughly 3,300 Python repositories—remains competitive on non-Python codebases, outperforming BM25 and codesage-small-v2 in both zero-shot and PEFT settings, demonstrating strong cross-language transfer.

Across datasets, SpIDER improves Recall@20 and Acc@20 of SweRankEmbed-Small by at least 13% and 14%, respectively, indicating that graph-aware retrieval yields substantial performance gains even when paired with already strong semantic embeddings.

Retrieval strategy is agnostic to downstream rerankers Our retrieval strategy is designed to operate independently of downstream reranking or localization components. To validate this property, we evaluate all retrieval methods in conjunction with an LLM-based reranker following Reddy et al. (2025), which selects the top-3 functions from $K = 20$ retrieved candidates based on its reasoning capabilities.

As shown in Table 2, the improved top-20 coverage provided by SpIDER consistently translates into higher Recall@3 and Acc@3 after reranking. This implies that gains achieved at the retrieval stage are preserved and often amplified by downstream reranking, and that SpIDER is complementary to, rather than coupled with, specific localization strategies.

Improved retrieval translates to better code generation To evaluate downstream impact, we pass retrieved function paths to swe-mini-agent (Yang et al., 2024) with Claude Sonnet 4.5, measuring pass rate on SWE-Bench-Verified at $K = 5$ and $K = 10$.

As shown in Figure 3, SpIDER yields consistent improvements across both context configurations, achieving gains

Table 4. **Ablation on BFS exploration depth (d)** using `SpIDER + SweRankEmbed-Small` for $K = 20$, $N = 100$, $C = 3$. Increasing d improves Recall@20 and Accuracy@20 across languages, while LLM input/output tokens scale with d.

d	Python				JavaScript				Java				TypeScript			
	Recall@20	Acc@20	Tokens Input	Tokens Output	Recall@20	Acc@20	Tokens Input	Tokens Output	Recall@20	Acc@20	Tokens Input	Tokens Output	Recall@20	Acc@20	Tokens Input	Tokens Output
2	0.45	0.36	2.6K	32	0.54	0.46	2.0K	50	0.33	0.25	2.2K	22	0.31	0.23	1.5K	21
4	0.46	0.37	3.6K	35	0.57	0.49	3.1K	25	0.33	0.25	2.1K	54	0.30	0.24	2.0K	24
6	0.50	0.41	11.9K	48	0.62	0.53	10.1K	42	0.33	0.25	3.6K	67	0.36	0.28	4.3K	33
8	0.55	0.44	29.7K	69	0.67	0.58	15.0K	48	0.34	0.27	5.6K	82	0.36	0.28	5.6K	37

of +0.46 percentage points (pp) at $K = 5$ and +2.74 pp at $K = 10$, corresponding to 2 and 12 additional resolved instances, respectively. These results demonstrate that enhanced retrieval coverage directly translates into measurable gains in downstream code generation performance, completing the chain from graph-aware retrieval to end-task success.

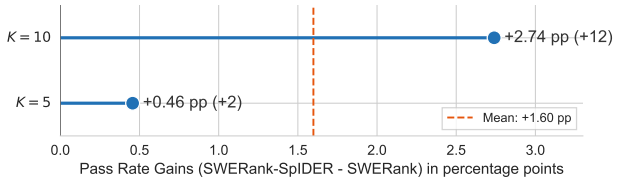

Figure 3. Pass rate improvement of `SWERankEmbed-Small-SpIDER` over `SWERank` on `SWE-bench Verified` with $C = 5, d = 4, N = 500$.

5.3. Ablation Experiments

To understand how `SpIDER`'s hyperparameters influence retrieval and to guide downstream users, we ablate four key parameters: number of seed centers C, retrieval budget K, BFS exploration depth d, and primary neighborhood filtering threshold N. For each, we describe the motivation, its effect on performance, trade-offs in computational cost/LLM usage, and tuning guidance for different codebases and resource budgets.

Number of Top C Centers The number of seed centers C determines how many top-ranked nodes are used as starting points for neighborhood exploration. Increasing C expands the explored neighborhood, adding more relevant neighbors to the top-K retrieved nodes but potentially displacing lower-ranked semantic candidates. Each neighborhood exploration requires one LLM call, so higher C increases total LLM invocations.

Table 3 shows that performance saturates at $C = 5$ for $N = 500$, $d = 4$, and $K = 20$, indicating that exploring neighbors of the top-ranked nodes efficiently recovers ground-truth functions while keeping LLM usage reasonable. Users can adjust C according to their LLM budget.

Retrieval Budget K The retrieval budget K controls the number of candidate nodes returned. As expected, performance improves monotonically with increasing K (Figures 2 and 6), since a larger pool increases the chance of including relevant nodes. Users can select K based on the constraints of downstream tasks, balancing retrieval quality against processing cost.

Exploration Depth d The maximum BFS depth d determines how many neighbors are explored around each seed center. Larger d increases the number of candidate neighbors, improving Recall@20 and Acc@20 (Table 4) but also increasing token consumption during the LLM-based neighborhood filtering. Users should select d considering the trade-off between retrieval gains and LLM resource usage.

Primary Neighborhood Filtering Threshold N The threshold N controls how many neighbors pass the initial semantic filter before LLM-based refinement. Higher N allows more neighbors to be considered, increasing the chance of recovering relevant nodes but also consuming more LLM tokens. Ablations in Table 11 show performance saturates around $N = 300$ for $C = 3$, $d = 4$, and $K = 20$. Like d, N can be tuned based on downstream LLM token budgets.

Takeaways Overall, these ablations show that `SpIDER`'s hyperparameters offer a clear trade-off between retrieval performance and LLM cost. Settings $C = 5$, $d = 4$, $N = 500$, $K = 20$ provide strong performance across `SWE-PolyBench`, while users can tune them according to resource constraints. Our results highlight that leveraging top-ranked nodes and their neighbors effectively maximizes retrieval coverage while controlling computational cost.

6. Conclusion

We introduced `SpIDER`, a neurosymbolic code retrieval framework built on dense embedding models and spatial information from graph representations of a codebase. Through rigorous empirical analysis, we demonstrated its efficacy and efficiency over existing methods in the challenging setting of function-level retrieval. `SpIDER`'s exploration depth can be efficiently controlled by adjusting the subgraph size during retrieval, maintaining robust performance even with compact subgraph configurations for token-constrained applications. While we pioneer retrieval evaluation for non-Python languages such as Java, JavaScript and TypeScript, `SpIDER` can be readily adapted to others as it builds on the `Tree-sitter` library.

References

Chen, Z., Tang, R., Deng, G., Wu, F., Wu, J., Jiang, Z., Prasanna, V., Cohan, A., and Wang, X. LocAgent: Graph-guided LLM agents for code localization. In Che, W., Nabende, J., Shutova, E., and Pilehvar, M. T. (eds.), *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 8697–8727, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.426. URL https://aclanthology.org/2025.acl-long.426/.

Chowdhury, N., Aung, J., Shern, C. J., Jaffe, O., Sherburn, D., Starace, G., Mays, E., Dias, R., Aljubeh, M., Glaese, M., Jimenez, C. E., Yang, J., Ho, L., Patwardhan, T., Liu, K., and Madry, A. Introducing SWE-bench Verified, 2024a. URL https://openai.com/index/introducing-swe-bench-verified/. Accessed on March 2, 2025.

Chowdhury, N., Aung, J., Shern, C. J., Jaffe, O., Sherburn, D., Starace, G., Mays, E., Dias, R., Aljubeh, M., Glaese, M., Jimenez, C. E., Yang, J., Liu, K., and Madry, A. Introducing SWE-bench Verified. OpenAI Blog, August 2024b. URL https://openai.com/index/introducing-swe-bench-verified/.

Fehr, F. J., Teja S, P., Franceschi, L., and Zappella, G. CoRet: Improved retriever for code editing. In Che, W., Nabende, J., Shutova, E., and Pilehvar, M. T. (eds.), *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pp. 775–789, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-252-7. doi: 10.18653/v1/2025.acl-short.62. URL https://aclanthology.org/2025.acl-short.62/.

Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., and Zhou, M. CodeBERT: A pre-trained model for programming and natural languages. In Cohn, T., He, Y., and Liu, Y. (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 1536–1547, Online, November 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.findings-emnlp.139. URL https://aclanthology.org/2020.findings-emnlp.139/.

Gauthier, P. Aider is ai pair programming in your terminal. https://github.com/paul-gauthier/aider, 2024.

Guo, D., Ren, S., Lu, S., Feng, Z., Tang, D., Liu, S., Zhou, L., Duan, N., Svyatkovskiy, A., Fu, S., Tufano, M., Deng, S. K., Clement, C., Drain, D., Sundaresan, N., Yin, J., Jiang, D., and Zhou, M. Graphcodebert: Pre-training code representations with data flow, 2021. URL https://arxiv.org/abs/2009.08366.

Hu, E. J., yelong shen, Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL https://openreview.net/forum?id=nZeVKeeFYf9.

Jiang, Z., Ren, X., Yan, M., Jiang, W., Li, Y., and Liu, Z. Cosil: Software issue localization via llm-driven code repository graph searching, 2025. URL https://arxiv.org/abs/2503.22424.

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. R. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=VTF8yNQM66.

Kang, S., An, G., and Yoo, S. A quantitative and qualitative evaluation of llm-based explainable fault localization. *Proc. ACM Softw. Eng.*, 1(FSE), July 2024. doi: 10.1145/3660771. URL https://doi.org/10.1145/3660771.

Luo, Q., Ye, Y., Liang, S., Zhang, Z., Qin, Y., Lu, Y., Wu, Y., Cong, X., Lin, Y., Zhang, Y., Che, X., Liu, Z., and Sun, M. Repoagent: An llm-powered open-source framework for repository-level code documentation generation, 2024.

Ouyang, S., Yu, W., Ma, K., Xiao, Z., Zhang, Z., Jia, M., Han, J., Zhang, H., and Yu, D. Repograph: Enhancing AI software engineering with repository-level code graph. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=dw9VUsSHGB.

Rashid, M. S., Bock, C., Zhuang, Y., Buchholz, A., Esler, T., Valentin, S., Franceschi, L., Wistuba, M., Sivaprasad, P. T., Kim, W. J., Deoras, A., Zappella, G., and Callot, L. Swe-polybench: A multi-language benchmark for repository level evaluation of coding agents, 2025. URL https://arxiv.org/abs/2504.08703.

Reddy, R. G., Suresh, T., Doo, J., Liu, Y., Nguyen, X. P., Zhou, Y., Yavuz, S., Xiong, C., Ji, H., and Joty, S. Swerank: Software issue localization with code ranking, 2025. URL https://arxiv.org/abs/2505.07849.

Robertson, S. E., Walker, S., Jones, S., Hancock-Beaulieu, M., and Gatford, M. Okapi at trec-3. In *Text Retrieval Conference*, 1994. URL https://api.semanticscholar.org/CorpusID:41563977.

Shaked, M. and Shanthikumar, J. G. *Stochastic Orders*. Springer, 2007.

Suresh, T., Reddy, R. G., Xu, Y., Nussbaum, Z., Mulyar, A., Duderstadt, B., and Ji, H. CoRNStack: High-quality contrastive data for better code retrieval and reranking.

In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=iyJOUELYir.

Xia, C. S., Deng, Y., Dunn, S., and Zhang, L. Agentless: Demystifying llm-based software engineering agents, 2024. URL https://arxiv.org/abs/2407.01489.

Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., and Press, O. Swe-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.

Zan, D., Huang, Z., Liu, W., Chen, H., Zhang, L., Xin, S., Chen, L., Liu, Q., Zhong, X., Li, A., Liu, S., Xiao, Y., Chen, L., Zhang, Y., Su, J., Liu, T., Long, R., Shen, K., and Xiang, L. Multi-swe-bench: A multilingual benchmark for issue resolving, 2025. URL https://arxiv.org/abs/2504.02605.

Zhang, D., Ahmad, W. U., Tan, M., Ding, H., Nallapati, R., Roth, D., Ma, X., and Xiang, B. CODE REPRESENTATION LEARNING AT SCALE. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=vfzRRjumpX.

Örwall, A. Moatless tools, 2024. URL https://github.com/aorwall/moatless-tools.

A. Appendix

A.1. Theoretical Analysis

The proof for Theorem 4.2 is simple. We start by providing a theoretical justification for why incorporating graph locality and LLM-based neighbor selection improves retrieval performance over standard dense embedding retrieval. Our analysis focuses on recall at a fixed budget K and relies on mild structural and statistical assumptions.

A.2. Modeling Assumptions

We follow the problem definition from Section 3 and use $\mathcal{S}_K$ to denote $\mathcal{S}_K(Q)$ for notational convenience. Then we make the following assumptions.

Assumption 1 (Score Distributions). There exist distributions F_1 and F_0 on $\mathbb{R}$ such that

$$s_Q(v) \sim \begin{cases} F_1, & v \in \mathcal{V}^*, \\ F_0, & v \notin \mathcal{V}^*, \end{cases}$$

where $F_i(t) = \Pr[s_Q(v) \le t \mid s_Q(v) \sim F_i]$ and F_1 first-order stochastically dominates F_0, i.e.,

$$F_1(t) \le F_0(t) \quad \forall t \in \mathbb{R}.$$

(See (Shaked & Shanthikumar, 2007) for background on stochastic dominance.)

Assumption 2 (Graph Locality of Relevance). There exists an integer $d \ge 1$ such that for any $v \in \mathcal{V}^*$,

$$\Pr\left(u \in \mathcal{V}^* \mid u \in \Gamma_d(\{v\})\right) \ge \Pr(u \in \mathcal{V}^*).$$

That is, relevance is non-negatively correlated with graph proximity.

Assumption 3 (LLM Selector Accuracy). For any $u \in \mathcal{C}_Q$,

$$\Pr(\mathcal{L}(u) = 1 \mid u \in \mathcal{V}^*) = \alpha$$
$$\Pr(\mathcal{L}(u) = 1 \mid u \notin \mathcal{V}^*) = \beta,$$

where $0 \le \beta < \alpha \le 1$.

A.3. Expected Recall Change

Define

$$B_Q = \mathcal{S}_K \setminus \mathcal{S}_{K-|\mathcal{U}_Q|}$$
$$\pi_B := \Pr(u \in \mathcal{V}^* \mid u \in B_Q),$$
$$\pi_\Gamma := \Pr(u \in \mathcal{V}^* \mid u \in \hat{\mathcal{C}}_Q \setminus \mathcal{S}_K).$$

Here, B_Q refers to bottom $|\mathcal{U}_Q|$ ranked nodes in $\mathcal{S}_K(Q)$.

Proposition A.1 (Expected Recall Difference)**.** *Under Assumptions 1–3, the expected change in recall satisfies*

$$\mathbb{E}[\mathrm{Rec@}K(\mathcal{S}_K^{\mathrm{LLM}})] - \mathbb{E}[\mathrm{Rec@}K(\mathcal{S}_K)] = \frac{\alpha\,\mathbb{E}[|\mathcal{U}_Q \cap \mathcal{V}^*|] - \mathbb{E}[|B_Q \cap \mathcal{V}^*|] - \beta\,\mathbb{E}[|\mathcal{U}_Q \setminus \mathcal{V}^*|]}{|\mathcal{V}^*|}.$$

Proof. By construction,

$$|\mathcal{S}_K^{\mathrm{LLM}} \cap \mathcal{V}^*| = |\mathcal{S}_K \cap \mathcal{V}^*| - |B_Q \cap \mathcal{V}^*| + |\mathcal{U}_Q \cap \mathcal{V}^*|.$$

Taking expectations and dividing by $|\mathcal{V}^*|$ yields the result. □

A.4. Sufficient Condition for Improvement

Proposition A.2 (Restating Sufficient Condition for Recall Improvement)**.** *Under the assumptions 1, 2 and 3 listed in Section A.2, if* $\alpha\pi_\Gamma > \pi_B + \beta(1-\pi_\Gamma)$*, then it holds that*

$$\mathbb{E}[\mathrm{Rec@}K(\mathcal{S}_K^{\mathrm{LLM}})] > \mathbb{E}[\mathrm{Rec@}K(\mathcal{S}_K)].$$

Proof. By linearity of expectation,

$$\mathbb{E}[|\mathcal{U}_Q \cap \mathcal{V}^*|] = \alpha K \pi_\Gamma, \quad \mathbb{E}[|\mathcal{U}_Q \setminus \mathcal{V}^*|] = K(1-\pi_\Gamma).$$

Similarly,

$$\mathbb{E}[|B_Q \cap \mathcal{V}^*|] = K\pi_B.$$

Substituting into Proposition A.1 and simplifying yields the stated claim. This concludes the proof for Theorem 4.2 □

A.5. Graph-Structured Relevance

We now show that π_Γ is strictly larger than π_B under a standard cluster assumption.

Definition (Relevant Subgraph). Let $H_q = (V_q, E_q)$ be an induced subgraph such that $\mathcal{V}^* \subseteq V_q$ and

$$\mathrm{diam}(H_q) \leq d.$$

Proposition A.3 (Graph Expansion Enriches Relevance)**.** *Suppose:*

1. *There exists at least one seed* $v \in C_q \cap V_q$.
2. *Assumptions 1 and 2 hold.*

Then

$$\pi_\Gamma \geq \Pr(u \in \mathcal{V}^*),$$

with strict inequality unless $\mathcal{V}^* = V_q$.

Proof. Since $\mathrm{diam}(H_q) \leq d$, all nodes in V_q lie in $\Gamma_d(\{v\})$. By Assumption 2, conditioning on membership in $\Gamma_d(\{v\})$ increases the probability of relevance. Restricting further to TopN preserves this enrichment by Assumption 1 (stochastic dominance). □

A.6. Conclusion

The above results show that when (i) relevant functions form a localized region in the code graph, (ii) dense retrieval places at least one seed in that region, and (iii) the LLM selector has higher true-positive than false-positive rate, then graph-aware retrieval strictly improves expected recall at fixed budget K.

A.7. Empirical Support for Assumption 2 (Graph Locality)

Proposition 4.2 relies on Assumption 2, which requires that relevance be non-negatively correlated with graph proximity, i.e., $\Pr(u \in \mathcal{V}^* \mid u \in \Gamma_d(\{v\})) \geq \Pr(u \in \mathcal{V}^*)$. To verify that this assumption is not merely a convenient modeling choice but actually holds on the benchmarks we evaluate on, we analyze the spatial distribution of ground-truth functions in `SWEBench-Verified`.

Per-instance distances. We restrict attention to instances with more than one ground-truth function edit, since locality is only meaningful when multiple targets are involved. Out of 500 total `SWEBench-Verified` instances, 193 contain multi-function edits. Among these 193 instances, 126 (65.3%) have all ground-truth functions within a maximum graph distance of $d = 4$ hops. The median per-instance *mean* distance between ground-truth functions is 2.20 hops along the `contains` edge type alone, and the median per-instance *maximum* distance is 3 hops.

Pairwise distances. At the pairwise level, we examine all 3,869 ground-truth function pairs across these 193 instances (each pair drawn from the same instance). Of these, 30% (1,171) are sibling functions in the same file or class at distance 2 hops, 45 pairs have distance 1 (nested containment), and 733 pairs are cross-class within the same file at distance 3–4. The remaining 48% (1,851) are cross-file or cross-package pairs with distance ≥ 6.

Implication for Proposition 4.2. A clear majority of multi-edit instances admit a small d that captures all ground-truth targets, and the bulk of within-instance ground-truth pairs concentrate at small hop counts. Both observations are direct empirical instantiations of Assumption 2: conditioning on membership in a small-radius neighborhood $\Gamma_d(\{v\})$ of a ground-truth node v raises the probability that a candidate u is itself ground truth, relative to the unconditional base rate over the codebase. The locality precondition of Proposition 4.2 therefore holds in practice, which in turn supports the practical relevance of the recall-improvement guarantee provided by SpIDER.

A.8. Algorithm

Algorithm 1 SpIDER

Require: Code graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, issue description Q, bi-modal encoder $\mathcal{F}(\cdot)$
Require: Parameters: retrieval budget K, filtering threshold N, number of centers C, search depth d
// Semantic Retrieval
Compute node embeddings: $\{\mathcal{F}(v)\}_{v \in \mathcal{V}}$
Compute query embedding: $\mathcal{F}(Q)$
Compute relevance scores: $s_Q(v) = \cos(\mathcal{F}(v), \mathcal{F}(Q))$ for all $v \in \mathcal{V}$
$\mathcal{S}_K(Q) \leftarrow \arg\operatorname{top}_K s_Q(v)$ top-K nodes ranked by $s_Q(v)$ {Baseline retrieval}
$\mathcal{S}_N(Q) \leftarrow$ top-N nodes ranked by $s_Q(v)$ {Candidate pool}
// Seed Selection
$\mathcal{C}_Q \leftarrow$ top-C nodes from $\mathcal{S}_K(Q)$ ranked by $s_Q(v)$ {Centers for BFS}
// Neighborhood Exploration
$\Gamma_d(\mathcal{C}_Q) \leftarrow \{u \in \mathcal{V} : \operatorname{dist}_{\mathcal{G}}(u, \mathcal{C}_Q) \leq d\}$ via BFS along '*contains*' edges
// Two-Stage Neighbor Filtering
$\hat{\mathcal{C}}_Q \leftarrow \Gamma_d(\mathcal{C}_Q) \cap \mathcal{S}_N(Q)$ {Stage 1: Semantic filtering}
$\mathcal{U}_Q \leftarrow \{u \in \hat{\mathcal{C}}_Q : \mathcal{L}(u) = 1\} \cup \mathcal{C}_Q$ {Stage 2: LLM selection}
// Output Construction
$\mathcal{S}_K^{\mathrm{LLM}}(Q) \leftarrow \mathcal{U}_Q \cup \mathcal{S}_{K-|\mathcal{U}_Q|}(Q)$ {where $\mathcal{S}_{K-|\mathcal{U}_Q|}(Q) = \arg\operatorname{top}_{\substack{K-|\mathcal{U}_Q| \\ v \in \mathcal{S}_K(\mathcal{Q})}} s_Q(v)$}
return $\mathcal{S}(Q)$

Note: The LLM selector $\mathcal{L}$ receives the source code content of all candidates $u \in \hat{\mathcal{C}}_Q$ for a given center, along with the issue description Q. It returns $\mathcal{L}(u) = 1$ for candidates deemed relevant to resolving the issue, and 0 otherwise.

A.9. Graph construction details

JavaScript did not have a unique class data type until 2015 as it was only introduced ECMAScript 2015 also known as ES6. Hence most of the functionality of classes were implemented through functions, causing increased variations in the definition styles of a function (e.g. a nested function with the definition of container function at some other location in the same/different file).

A.10. Extension to Class-level edits

In Table 5, we demonstrate the efficacy of the proposed approach at a class level. Here, we include only Python and Java here, excluding JavaScript, as most JavaScript repositories in the SWE-PolyBench dataset lack significant use of class syntax (introduced in ECMAScript 2015), resulting in very few instances with class ground truth nodes. A ground truth edit is classified as a class-level edit (and the corresponding class as a ground truth class node), if the patch introduces a modification within any part of the class declaration or definition (e.g., constructors, destructors, or methods) to address the issue description.

Table 5. Class-level performance on SWE-PolyBench for $K = 20$, $N = 500$, $C = 5$, $d = 2$. Winners per metric are in **bold**.

Language	Model	Retrieval	Recall@20	Accuracy@20	MRR@20
Python	SweRankEmbed-Small ZS	SpIDER	**0.65**	**0.54**	0.36
		DER	0.59	0.48	**0.45**
	CodeSAGE-Small ZS	SpIDER	**0.39**	**0.33**	0.16
		DER	0.33	0.25	**0.19**
	CodeSAGE-Small-PEFT	SpIDER	**0.31**	**0.26**	0.16
		DER	0.30	0.22	**0.19**
	BM25	SpISR	**0.60**	**0.52**	0.28
		SR	0.58	0.48	**0.35**
Java	SweRankEmbed-Small ZS	SpIDER	**0.48**	**0.37**	**0.34**
		DER	0.46	**0.37**	**0.34**
	CodeSAGE-Small ZS	SpIDER	**0.25**	**0.17**	**0.11**
		DER	0.23	**0.17**	0.10
	CodeSAGE-Small-PEFT	SpIDER	0.26	0.18	0.12
		DER	**0.28**	**0.21**	**0.14**
	BM25	SpISR	**0.33**	**0.26**	0.22
		SR	**0.33**	**0.26**	**0.25**

A.11. Additional Experiments

Table 6. Performance of SpIDER + SweRankEmbed-Small ZS encoder for different numbers of centers with $K = 20$, $N = 100$, $C = 3$

d	Python			JavaScript			Java			TypeScript		
	Recall@20	Accuracy@20	MRR@20	Recall@20	Accuracy@20	MRR@20	Recall@20	Accuracy@20	MRR@20	Recall@20	Accuracy@20	MRR@20
2	0.45	0.36	0.28	0.54	0.46	0.17	0.33	0.25	0.18	0.31	0.23	0.06
4	0.46	0.37	0.30	0.57	0.49	0.18	0.33	0.25	0.18	0.30	0.24	0.06
6	0.50	0.41	0.33	0.62	0.53	0.20	0.33	0.25	0.18	0.36	0.28	0.07
8	0.55	0.44	0.38	0.67	0.58	0.22	0.34	0.27	0.20	0.36	0.27	0.07

Table 7. SpIDER + SweRankEmbed-Small ZS encoder performance with varying neighbor filtering threshold N for $K = 20$, $C = 3$, $d = 4$

N	Python			JavaScript			Java			TypeScript		
	Recall@20	Accuracy@20	MRR@20	Recall@20	Accuracy@20	MRR@20	Recall@20	Accuracy@20	MRR@20	Recall@20	Accuracy@20	MRR@20
50	0.45	0.37	0.29	0.56	0.48	0.18	0.32	0.24	0.19	0.31	0.24	0.06
100	0.46	0.37	0.30	0.57	0.49	0.18	0.33	0.25	0.18	0.30	0.24	0.06
300	0.48	0.39	0.30	0.58	0.50	0.18	0.35	0.26	0.19	0.32	0.25	0.06
500	0.49	0.39	0.30	0.59	0.51	0.18	0.35	0.26	0.19	0.31	0.24	0.06
700	0.48	0.39	0.31	0.59	0.51	0.18	0.35	0.25	0.18	0.31	0.23	0.06
900	0.48	0.39	0.31	0.58	0.50	0.18	0.34	0.25	0.19	0.31	0.24	0.06
1000	0.48	0.39	0.31	0.59	0.51	0.18	0.36	0.27	0.19	0.32	0.24	0.06

MRR@K Gains Depend on Graph Neighborhoods The MRR@K metric benefits from SpIDER primarily when the first relevant ground-truth item within the top-K results is a neighbor of a center node. In cases where the first relevant item is a non-center node, particularly in TypeScript as shown in Tab. 2—SpIDER may rank neighbors of center nodes ahead, slightly reducing MRR@K. Since SpIDER is not explicitly designed to optimize the precise ranking order within the top-K results, improvements in MRR@K over baseline DER methods are modest but still observable.

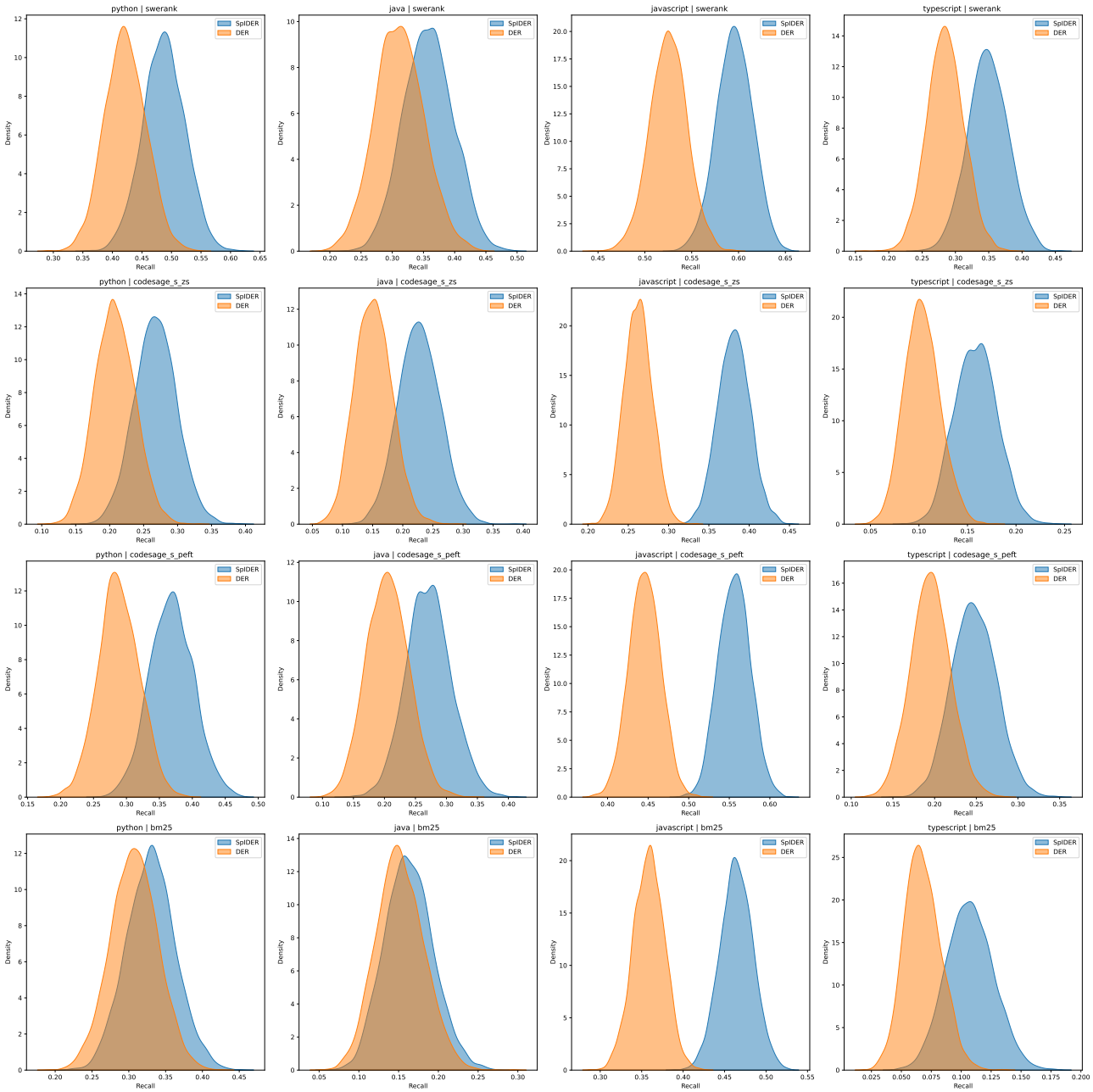

Figure 4. KDE (Kernel Density Estimate) plots with bootstrapped results of SWE-PolyBench benchmark for Recall@20 performance across various retrieval methods.

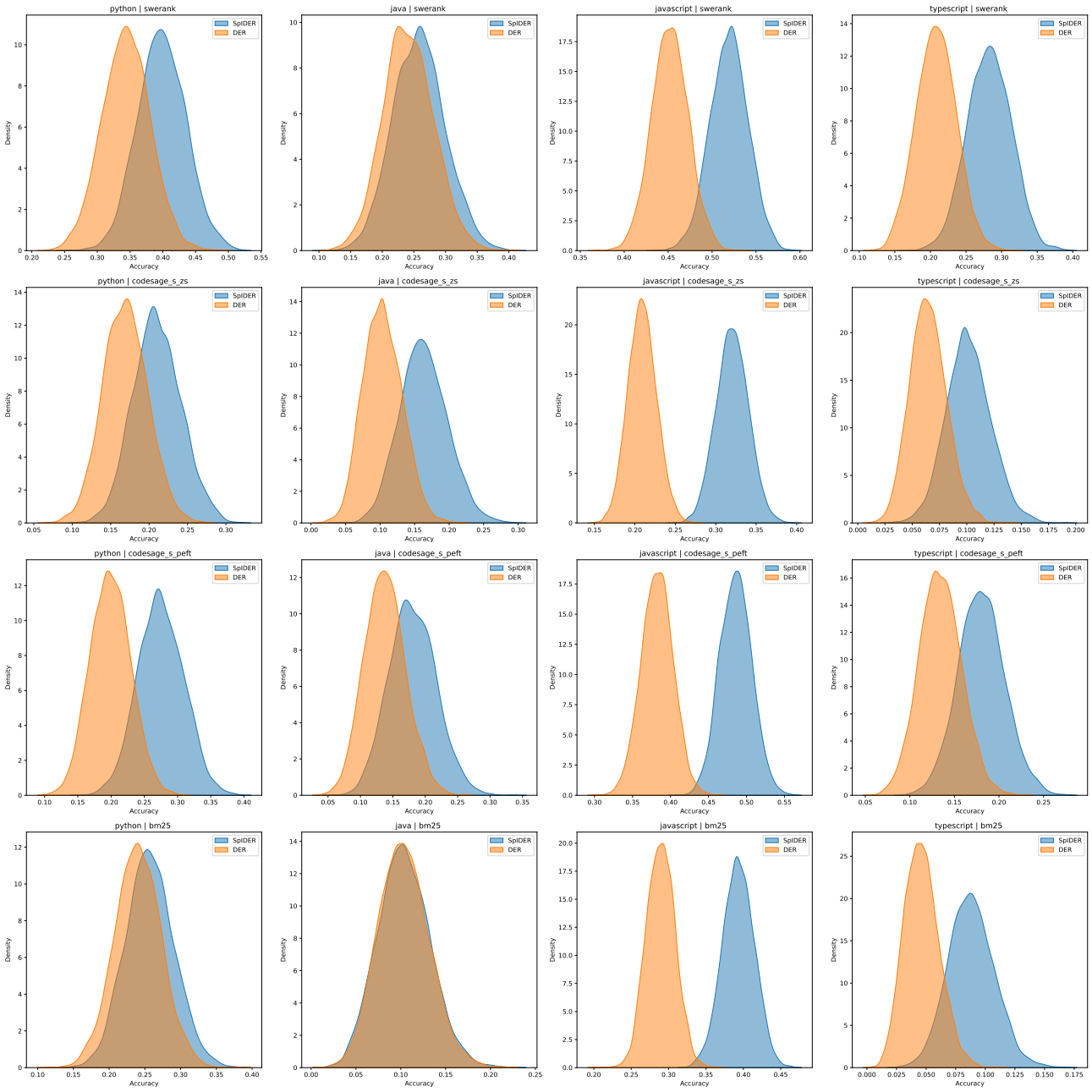

Figure 5. KDE (Kernel Density Estimate) plots with bootstrapped results of SWE-PolyBench benchmark for Acc@20 performance across various retrieval methods.

Table 8. LocAgent performance on SWEBench-Verified and a subset of Python instances in SWE-PolyBench. Running LocAgent took over 48 hours on SWEBench-Verified and approximately 24 hours on SWE-PolyBench, highlighting a key limitation of this approach. In addition to the long runtime, LocAgent also requires significantly more LLM invocations per instance compared to dense embedding retrieval and BM25, resulting in higher computational costs.

Dataset	Metrics	K											
		3	5	10	20	30	40	50	60	70	80	90	100
SWEBench-Verified	**Recall@K**	0.18	0.26	0.41	0.54	0.62	0.68	0.69	0.71	0.72	0.73	0.74	0.74
	Acc@K	0.17	0.23	0.37	0.48	0.57	0.61	0.63	0.64	0.66	0.66	0.67	0.67
	MRR@K	0.12	0.14	0.16	0.17	0.17	0.17	0.17	0.17	0.17	0.17	0.18	0.18
SWE-PolyBench (Python subset)	**Recall@K**	0.23	0.30	0.41	0.49	0.53	0.58	0.58	0.59	0.59	0.60	0.61	0.61
	Acc@K	0.17	0.23	0.31	0.37	0.41	0.46	0.46	0.46	0.46	0.47	0.48	0.49
	MRR@K	0.21	0.23	0.25	0.25	0.25	0.26	0.26	0.26	0.26	0.26	0.26	0.26

Table 9. Comparison of LocAgent with dense retrieval methods + LLM reranker for Python repositories in SWE-PolyBench

Dataset	Model	Recall@3	Accuracy@3	MRR@3
SWE-PolyBench	LocAgent	0.18	0.17	0.12
	SweRankEmbed-Small ZS+ SpIDER + LLM reranker	0.44	0.36	0.54
	CodeSAGE-Small ZS + SpIDER + LLM reranker	0.23	0.18	0.31
	CodeSAGE-Small-PEFT + SpIDER + LLM reranker	0.32	0.25	0.42
	BM25+ SpISR	0.30	0.23	0.38

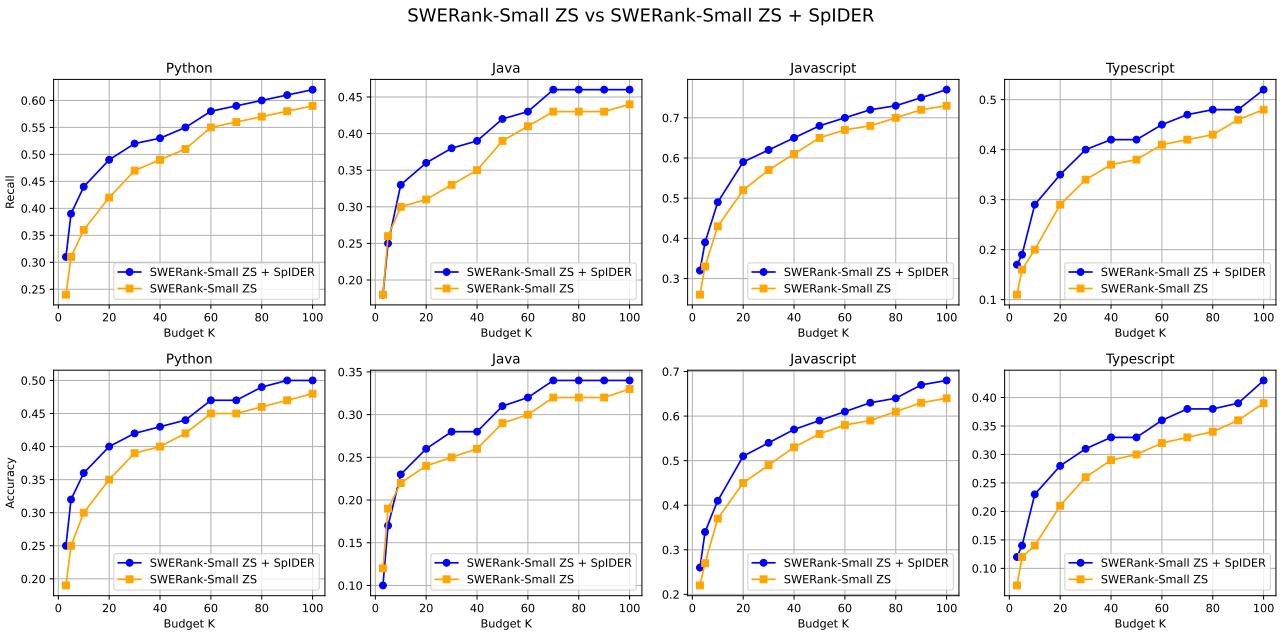

Figure 6. SpIDER vs DER performance at various values of K on SWE-PolyBench benchmark for SweRankEmbed-Small ZS embedding model, $N = 500$, $C = 5$, $d = 4$.

Table 10. SWE-PolyBench retrieval performance with 95% confidence intervals ($\mu \pm$ CI) for $K = 20$, $N = 500$, $C = 5$, $d = 4$

Language	Model	Retrieval	Recall@20	Accuracy@20
Python	SweRankEmbed-Small ZS	SpIDER	0.49 ± 0.06	0.40 ± 0.07
		DER	0.42 ± 0.06	0.35 ± 0.07
	CodeSAGE-Small ZS	SpIDER	0.27 ± 0.06	0.21 ± 0.06
		DER	0.21 ± 0.06	0.17 ± 0.06
	CodeSAGE-Small-PEFT	SpIDER	0.37 ± 0.06	0.27 ± 0.07
		DER	0.29 ± 0.06	0.20 ± 0.06
	BM25	SpISR	0.33 ± 0.07	0.26 ± 0.07
		SR	0.31 ± 0.06	0.24 ± 0.07
Java	SweRankEmbed-Small ZS	SpIDER	0.36 ± 0.08	0.27 ± 0.08
		DER	0.31 ± 0.07	0.24 ± 0.08
	CodeSAGE-Small ZS	SpIDER	0.23 ± 0.07	0.17 ± 0.07
		DER	0.15 ± 0.06	0.10 ± 0.05
	CodeSAGE-Small-PEFT	SpIDER	0.28 ± 0.08	0.18 ± 0.07
		DER	0.21 ± 0.07	0.14 ± 0.06
	BM25	SpISR	0.16 ± 0.06	0.10 ± 0.05
		SR	0.15 ± 0.06	0.10 ± 0.05
JavaScript	SweRankEmbed-Small ZS	SpIDER	0.60 ± 0.04	0.52 ± 0.04
		DER	0.52 ± 0.04	0.45 ± 0.04
	CodeSAGE-Small ZS	SpIDER	0.38 ± 0.04	0.32 ± 0.04
		DER	0.26 ± 0.04	0.21 ± 0.04
	CodeSAGE-Small-PEFT	SpIDER	0.55 ± 0.04	0.48 ± 0.05
		DER	0.45 ± 0.04	0.38 ± 0.04
	BM25	SpISR	0.46 ± 0.04	0.39 ± 0.04
		SR	0.36 ± 0.04	0.29 ± 0.04
TypeScript	SweRankEmbed-Small ZS	SpIDER	0.35 ± 0.06	0.28 ± 0.05
		DER	0.29 ± 0.05	0.21 ± 0.05
	CodeSAGE-Small ZS	SpIDER	0.16 ± 0.04	0.10 ± 0.04
		DER	0.10 ± 0.04	0.06 ± 0.04
	CodeSAGE-Small-PEFT	SpIDER	0.25 ± 0.05	0.18 ± 0.05
		DER	0.19 ± 0.05	0.13 ± 0.05
	BM25	SpISR	0.11 ± 0.04	0.09 ± 0.04
		SR	0.07 ± 0.03	0.05 ± 0.03

Table 11. **Ablation on primary neighborhood filtering threshold** N using SpIDER + SweRankEmbed-Small for $K = 20$, $C = 3$, $d = 4$. Increasing N improves coverage and Recall@20 with modest increases in LLM token usage. Avg. tokens indicate average consumption per LLM call; number of calls equals C. Best Recall and Accuracy per language are highlighted.

N	Python			JavaScript			Java			TypeScript		
	Recall	Accuracy	Tokens (Input / Output)	Recall	Accuracy	Tokens (Input / Output)	Recall	Accuracy	Tokens (Input / Output)	Recall	Accuracy	Tokens (Input / Output)
50	0.45	0.37	3108 / 33	0.56	0.48	2383 / 23	0.32	0.24	1845 / 49	0.31	0.24	1763 / 23
100	0.46	0.37	3591 / 35	0.57	0.49	3115 / 25	0.33	0.25	2083 / 54	0.30	0.24	1994 / 24
300	0.48	0.39	4848 / 37	0.58	0.50	5119 / 29	0.35	0.26	2644 / 56	0.32	0.25	2459 / 26
500	0.49	0.39	5708 / 39	0.59	0.51	6775 / 30	0.35	0.26	2978 / 57	0.31	0.24	2687 / 26
700	0.48	0.39	6361 / 39	0.59	0.51	7998 / 31	0.35	0.25	3203 / 59	0.31	0.23	2843 / 27
900	0.48	0.39	6903 / 40	0.58	0.50	8780 / 32	0.35	0.26	3343 / 58	0.31	0.24	2943 / 27
1000	0.48	0.39	7180 / 40	0.59	0.51	9098 / 32	0.36	0.27	3441 / 59	0.32	0.25	3001 / 27

Table 12. Effect of retrieval budget K on SpIDER + SweRankEmbed-Small performance for $N = 500$, $C = 5$, $d = 4$. Increasing K consistently improves Recall@K and Accuracy@K, while MRR@K remains stable. Best values per metric are highlighted.

K	Python			Java			JavaScript			TypeScript		
	Recall	Accuracy	MRR	Recall	Accuracy	MRR	Recall	Accuracy	MRR	Recall	Accuracy	MRR
3	0.31	0.25	0.27	0.17	0.09	0.17	0.32	0.26	0.16	0.17	0.12	0.06
5	0.39	0.32	0.30	0.24	0.16	0.18	0.39	0.34	0.17	0.19	0.14	0.06
10	0.44	0.36	0.30	0.33	0.23	0.19	0.49	0.41	0.17	0.29	0.23	0.06
20	0.47	0.39	0.29	0.35	0.26	0.19	0.59	0.51	0.18	0.35	0.28	0.06
30	0.52	0.42	0.30	0.37	0.27	0.19	0.62	0.54	0.18	0.40	0.31	0.06
40	0.52	0.43	0.30	0.38	0.28	0.19	0.65	0.57	0.18	0.42	0.33	0.07
50	0.54	0.44	0.30	0.42	0.31	0.19	0.68	0.59	0.18	0.42	0.33	0.06
60	0.58	0.47	0.31	0.43	0.32	0.19	0.70	0.61	0.18	0.45	0.36	0.06
70	0.58	0.46	0.30	0.45	0.34	0.19	0.72	0.63	0.18	0.47	0.38	0.07
80	0.60	0.48	0.30	0.46	0.34	0.19	0.73	0.64	0.18	0.48	0.38	0.07
90	0.61	0.50	0.30	0.46	0.34	0.19	0.75	0.67	0.18	0.48	0.39	0.06
100	0.62	0.50	0.30	0.46	0.34	0.19	0.77	0.68	0.18	0.52	0.43	0.07

Table 13. SweRankEmbed-Small zero-shot performance across varying retrieval budgets K for $N = 500$, $C = 5$, $d = 4$. Best Recall and Accuracy per language are highlighted.

K	Python			Java			JavaScript			TypeScript		
	Recall	Accuracy	MRR	Recall	Accuracy	MRR	Recall	Accuracy	MRR	Recall	Accuracy	MRR
3	0.24	0.19	0.27	0.18	0.12	0.24	0.26	0.22	0.27	0.11	0.07	0.15
5	0.31	0.25	0.29	0.26	0.19	0.26	0.33	0.27	0.28	0.16	0.12	0.16
10	0.36	0.30	0.30	0.30	0.22	0.26	0.43	0.37	0.30	0.20	0.14	0.17
20	0.42	0.35	0.30	0.31	0.24	0.26	0.52	0.45	0.30	0.29	0.21	0.18
30	0.47	0.39	0.30	0.33	0.25	0.26	0.57	0.49	0.31	0.34	0.26	0.18
40	0.49	0.40	0.30	0.35	0.26	0.27	0.61	0.53	0.31	0.37	0.29	0.18
50	0.51	0.42	0.31	0.39	0.29	0.27	0.65	0.56	0.31	0.38	0.30	0.18
60	0.55	0.45	0.31	0.41	0.30	0.27	0.67	0.58	0.31	0.41	0.32	0.18
70	0.56	0.45	0.31	0.43	0.32	0.27	0.68	0.59	0.31	0.42	0.33	0.18
80	0.57	0.46	0.31	0.43	0.32	0.27	0.70	0.61	0.31	0.43	0.34	0.18
90	0.58	0.47	0.31	0.43	0.32	0.27	0.72	0.63	0.31	0.46	0.36	0.18
100	0.59	0.48	0.31	0.44	0.33	0.27	0.73	0.64	0.31	0.48	0.39	0.18

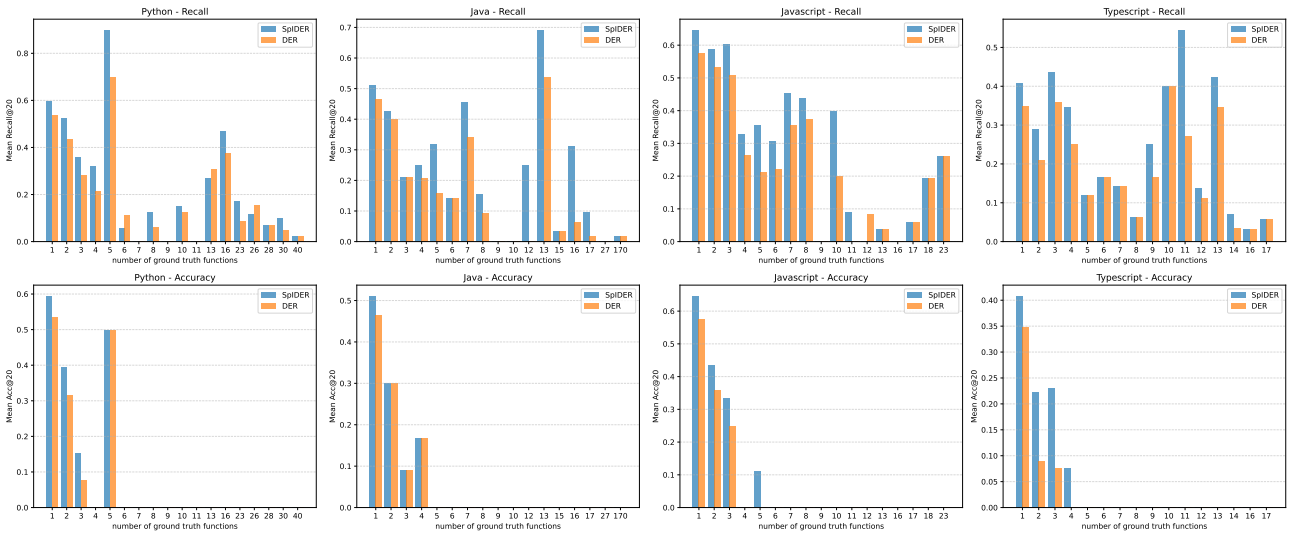

Figure 7. Comparing retrieval Recall@20 and Accuracy vs. number of edits for DER vs SpIDER on SweRankEmbed-Small ZS using SWE-PolyBench across various languages for $N = 500$, $C = 5$, $d = 4$. Top row: Recall@20. Bottom row: Accuracy.

Table 14. Comparison of dense retrieval and dense retrieval + LLM reranker for $K = 20$ and $N = 500$, $C = 5$, $d = 4$ on SWE-PolyBench. Winners per metric per embedding model are highlighted in light blue.

Language	Model	Retrieval	Dense Retrieval Only			Dense + LLM reranker		
			Recall@20	Accuracy@20	MRR@20	Recall@3	Accuracy@3	MRR@3
Python	SweRankEmbed-Small ZS	SpIDER	0.49	0.40	0.30	0.44	0.36	0.54
		DER	0.42	0.35	0.26	0.38	0.31	0.48
	CodeSAGE-Small ZS	SpIDER	0.27	0.21	0.13	0.23	0.18	0.31
		DER	0.21	0.17	0.11	0.19	0.16	0.25
	CodeSAGE-Small-PEFT	SpIDER	0.37	0.27	0.24	0.32	0.25	0.42
		DER	0.29	0.20	0.21	0.26	0.20	0.38
	BM25	SpISR	0.33	0.26	0.19	0.30	0.23	0.38
		SR	0.31	0.24	0.18	0.27	0.22	0.35
Java	SweRankEmbed-Small ZS	SpIDER	0.36	0.27	0.18	0.25	0.17	0.36
		DER	0.31	0.24	0.18	0.24	0.16	0.33
	CodeSAGE-Small ZS	SpIDER	0.23	0.17	0.10	0.16	0.09	0.26
		DER	0.15	0.10	0.07	0.11	0.07	0.19
	CodeSAGE-Small-PEFT	SpIDER	0.28	0.18	0.15	0.20	0.14	0.28
		DER	0.21	0.14	0.13	0.17	0.11	0.25
	BM25	SpISR	0.16	0.10	0.10	0.12	0.08	0.20
		SR	0.15	0.10	0.10	0.12	0.09	0.19
JavaScript	SweRankEmbed-Small ZS	SpIDER	0.60	0.52	0.18	0.43	0.37	0.43
		DER	0.52	0.45	0.16	0.41	0.34	0.42
	CodeSAGE-Small ZS	SpIDER	0.38	0.32	0.09	0.30	0.25	0.31
		DER	0.26	0.21	0.07	0.22	0.17	0.24
	CodeSAGE-Small-PEFT	SpIDER	0.55	0.48	0.15	0.41	0.35	0.41
		DER	0.45	0.38	0.14	0.36	0.30	0.39
	BM25	SpISR	0.46	0.39	0.13	0.36	0.30	0.38
		SR	0.36	0.29	0.11	0.31	0.25	0.34
TypeScript	SweRankEmbed-Small ZS	SpIDER	0.35	0.28	0.06	0.27	0.21	0.29
		DER	0.29	0.21	0.18	0.25	0.19	0.30
	CodeSAGE-Small ZS	SpIDER	0.16	0.10	0.03	0.14	0.09	0.19
		DER	0.10	0.06	0.06	0.09	0.06	0.14
	CodeSAGE-Small-PEFT	SpIDER	0.25	0.18	0.04	0.22	0.16	0.27
		DER	0.19	0.13	0.11	0.18	0.12	0.24
	BM25	SpISR	0.11	0.09	0.02	0.10	0.07	0.11
		SR	0.07	0.05	0.04	0.07	0.05	0.09

Table 15. Dense retrieval performance across benchmarks for $K = 20$, $N = 500$, $C = 5$, $d = 4$. Winners per metric per embedding model highlighted in light blue.

Language	Model	Retrieval	Recall@20	Accuracy@20	MRR@20
Multi-SWEBench					
Java	SweRankEmbed-Small ZS	SpIDER	0.37	0.28	0.08
		DER	0.32	0.24	0.08
	CodeSAGE-Small ZS	SpIDER	0.21	0.19	0.05
		DER	0.15	0.13	0.03
	CodeSAGE-Small-PEFT	SpIDER	0.31	0.25	0.10
		DER	0.23	0.17	0.09
	BM25	SpISR	0.23	0.17	0.07
		SR	0.22	0.17	0.05
JavaScript	SweRankEmbed-Small ZS	SpIDER	0.39	0.30	0.18
		DER	0.31	0.23	0.15
	CodeSAGE-Small ZS	SpIDER	0.17	0.12	0.08
		DER	0.09	0.06	0.05
	CodeSAGE-Small-PEFT	SpIDER	0.32	0.24	0.13
		DER	0.25	0.18	0.11
	BM25	SpISR	0.21	0.14	0.08
		SR	0.14	0.10	0.06
TypeScript	SweRankEmbed-Small ZS	SpIDER	0.52	0.42	0.08
		DER	0.40	0.32	0.23
	CodeSAGE-Small ZS	SpIDER	0.33	0.26	0.04
		DER	0.26	0.21	0.13
	CodeSAGE-Small-PEFT	SpIDER	0.30	0.26	0.05
		DER	0.19	0.15	0.12
	BM25	SpISR	0.32	0.27	0.04
		SR	0.24	0.21	0.11
SWEBench-Verified					
Python	SweRankEmbed-Small ZS	SpIDER	0.61	0.56	0.32
		DER	0.54	0.49	0.29
	CodeSAGE-Small ZS	SpIDER	0.30	0.27	0.13
		DER	0.21	0.19	0.10
	CodeSAGE-Small-PEFT	SpIDER	0.55	0.49	0.27
		DER	0.43	0.38	0.22
	BM25	SpISR	0.45	0.41	0.25
		SR	0.38	0.34	0.22

Table 16. **SpIDER with Qwen-Coder-30B-A3B-Instruct as the LLM.** Retrieval performance on SWE-PolyBench using Qwen-Coder-30B-A3B-Instruct as the language model for neighborhood filtering in SpIDER and for LLM reranking. Settings match Table 2: $K = 20$, $N = 500$, $C = 5$, $d = 4$. SpISR denotes the spatially informed sparse retriever analogous to SpIDER.

Language	Model	Retrieval	SWE-PolyBench	
			Retrieval Only	
			Recall@20	Accuracy@20
Python	SweRankEmbed-Small ZS	SpIDER	0.34	0.26
Java	SweRankEmbed-Small ZS	SpIDER	0.32	0.18
JavaScript	SweRankEmbed-Small ZS	SpIDER	0.60	0.46
TypeScript	SweRankEmbed-Small ZS	SpIDER	0.31	0.21

Table 17. Impact of the number of center nodes (C) on retrieval performance using SpIDER + SweRankEmbed-Small. Increasing C generally improves Recall@20 and Acc@20 across languages, while MRR@20 shows modest variation.

C	Python			JavaScript			Java			TypeScript		
	Recall@20	Acc@20	MRR@20	Recall@20	Acc@20	MRR@20	Recall@20	Acc@20	MRR@20	Recall@20	Acc@20	MRR@20
1	0.47	0.38	0.29	0.56	0.48	0.18	0.32	0.24	0.18	0.32	0.24	0.06
3	0.49	0.39	0.30	0.59	0.51	0.18	0.35	0.26	0.19	0.31	0.24	0.06
5	0.49	0.40	0.30	0.60	0.52	0.18	0.36	0.27	0.18	0.35	0.28	0.06
7	0.49	0.39	0.30	0.60	0.52	0.18	0.36	0.27	0.19	0.37	0.30	0.06
9	0.48	0.39	0.30	0.60	0.53	0.19	0.37	0.27	0.19	0.39	0.31	0.06